Arquitectura de Microservicios con Contenedores

Docentes:

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

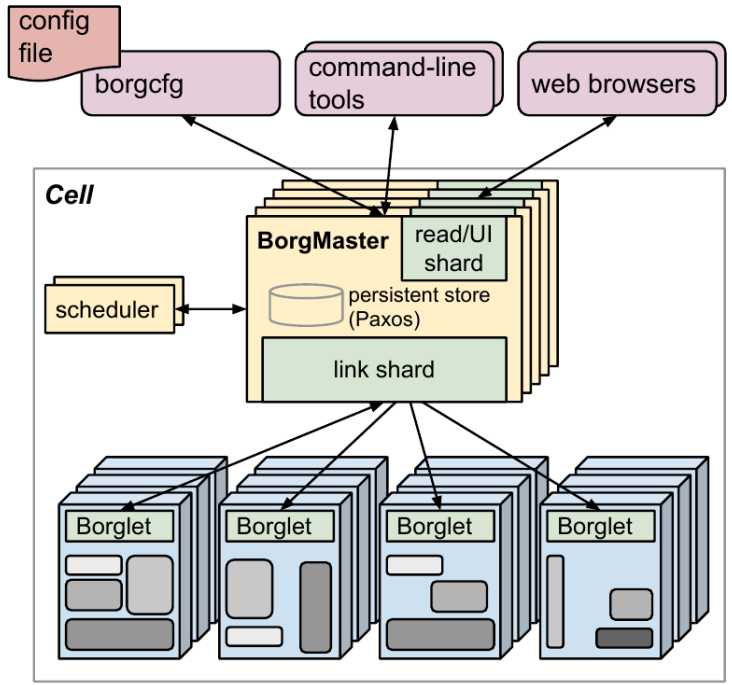
Kubernetes (K8S)

Historia

Desarrollado inicialmente por Google, inspirado en Borg, su sistema interno de administración de contenedores.

Fue el primer proyecto alojado por la Cloud Native Computing Foundation (CNCF), parte de la Linux Foundation.

Kubernetes es la palabra griega para piloto o timonel del barco.

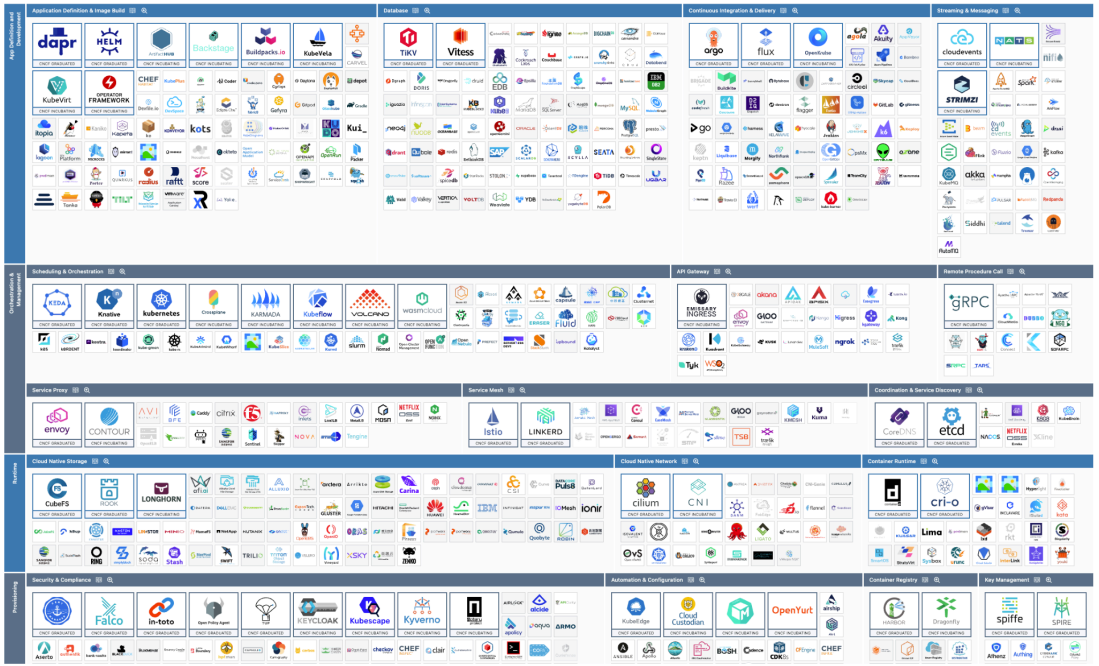


Arquitectura de Borg · Google, 2015

Fuente: CNCF · primer proyecto

Fuente: Google Research · Borg (2015), figura 1

Software en CNCF (panorama histórico)



Captura histórica: consultá el panorama actualizado en CNCF Landscape.

Fuente: CNCF Landscape · panorama actual

¿Qué hace Kubernetes?

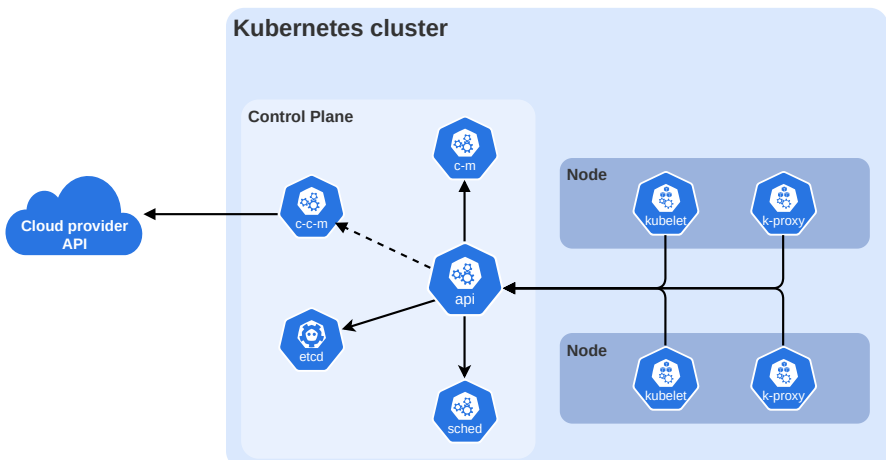
Kubernetes automatiza las tareas operativas de la administración de contenedores: despliega aplicaciones, coordina sus actualizaciones y mantiene la cantidad deseada de réplicas.

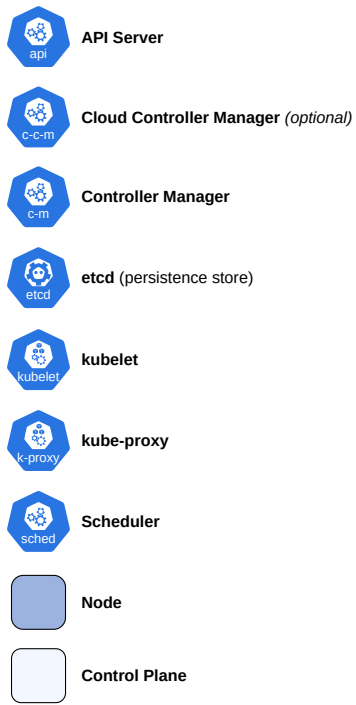
También permite configurar escalado automático y comprobar el estado de las aplicaciones. Las métricas, el almacenamiento de logs y otras funciones requieren componentes adicionales.

OPERACIONES AUTOMATIZADAS	ABSTRACCIÓN DE LA INFRAESTRUCTURA	ESTADO DE LAS APLICACIONES

Arquitectura de K8S

Un clúster tiene un **plano de control** que coordina el estado deseado y **nodos de trabajo** que ejecutan los Pods.





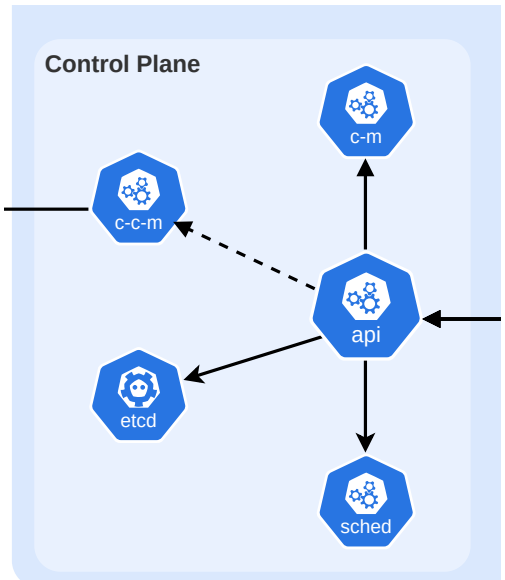
Fuente: Kubernetes · componentes

Plano de control

API server: expone la API de Kubernetes. Clientes y componentes coordinan el estado del clúster mediante esta API.

Scheduler: elige un nodo para cada Pod sin asignación y registra la decisión en la API.

Controller manager: ejecuta controladores que reconcilian el estado observado con el deseado.



Fuente: Kubernetes · plano de control

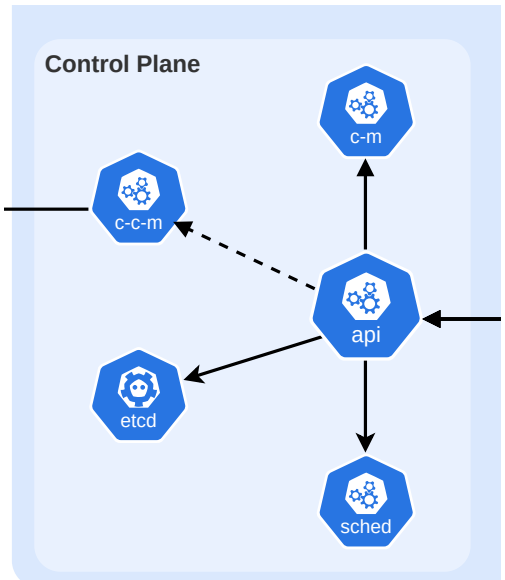
Estado y herramientas

etcd: persiste los datos de la API. Su disponibilidad y respaldo son críticos.

kubectl: cliente de línea de comandos para consultar y modificar recursos mediante la API.

kubeadm: inicializa clústeres, incorpora nodos y asiste en su actualización.

kubectl y kubeadm son herramientas, no componentes permanentes del plano de control.



Fuente: Kubernetes · kubeadm

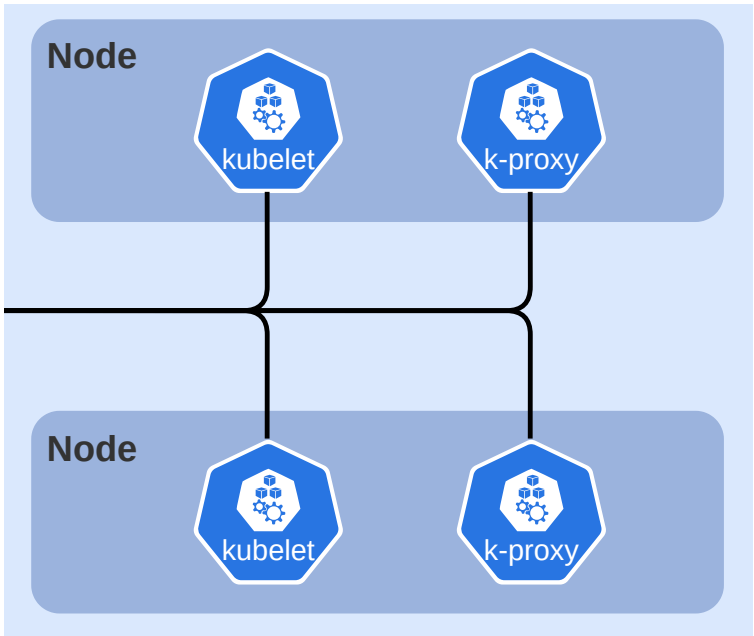
Nodo de trabajo

kubelet: observa los Pods asignados mediante la API y mantiene sus contenedores en ejecución.

Runtime CRI: ejecuta contenedores; por ejemplo, containerd o CRI-O.

Plugin CNI: configura la conectividad de los Pods.

kube-proxy: implementa reglas para Services; algunas soluciones de red lo reemplazan.



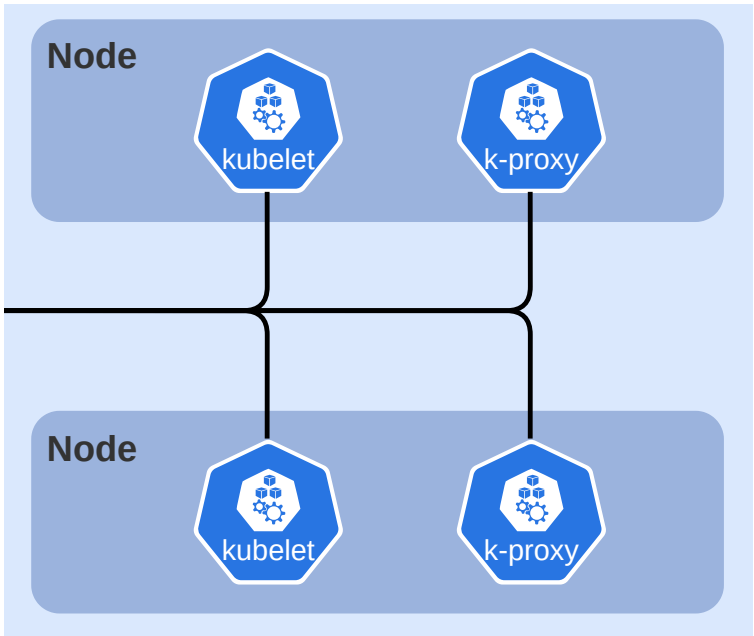
Fuente: Kubernetes · runtimes CRI

Pods en un nodo

El **Pod** es la unidad mínima que Kubernetes crea y administra. Agrupa uno o más contenedores que se ejecutan juntos en un mismo nodo.

El scheduler asigna el Pod a un nodo; el kubelet de ese nodo lo ejecuta.

Un Pod no migra entre nodos. Si hace falta reemplazarlo, un controlador crea otro Pod, con una nueva identidad (UID).



Fuente: Kubernetes · vida de un Pod

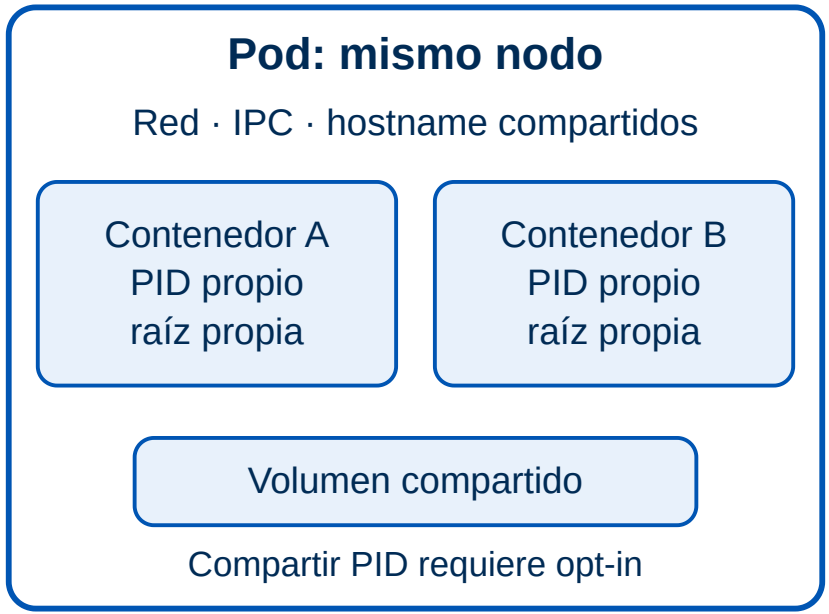
Workloads

Pods

Un Pod reúne contenedores de una misma aplicación que necesitan trabajar estrechamente juntos.

Comparten recursos de red y pueden montar volúmenes comunes. La especificación describe cómo ejecutar cada contenedor.

El caso más habitual es **un contenedor por Pod**. Agregar más contenedores tiene sentido cuando necesitan ese contexto compartido.



Fuente: Kubernetes · Pods

Contenedores en Linux

Un contenedor ejecuta procesos sobre el kernel del nodo.

- Los **namespaces de Linux** aíslan vistas del sistema: procesos, red y montajes, entre otras.
- Los **cgroups** controlan el uso de recursos, como CPU y memoria.
- El filesystem raíz proviene de la imagen, con una capa escribible propia.

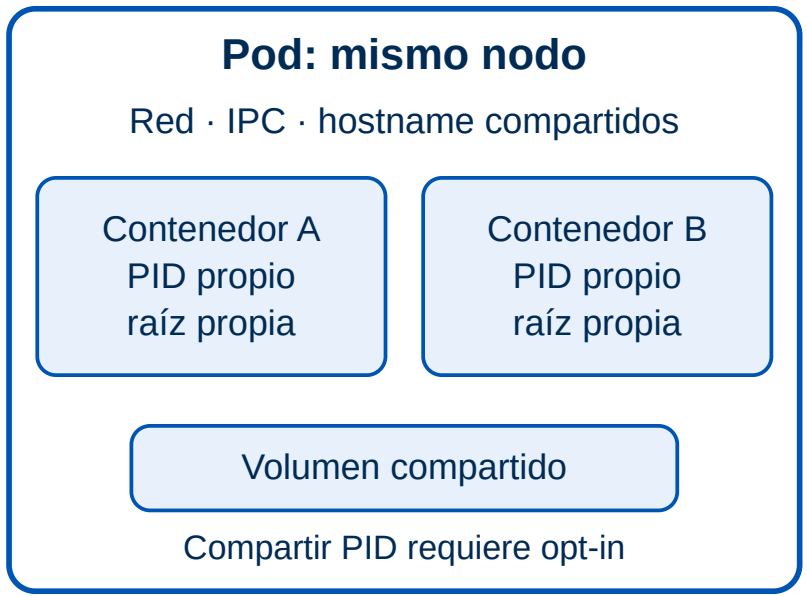
Estos mecanismos son complementarios: aislamiento y control de recursos.

Los namespaces del kernel no son los objetos **Namespace** de la API de Kubernetes.

Fuente: Kubernetes · contenedores

Pod en Kubernetes

Los contenedores comparten parte del contexto del Pod y conservan aislamiento propio.

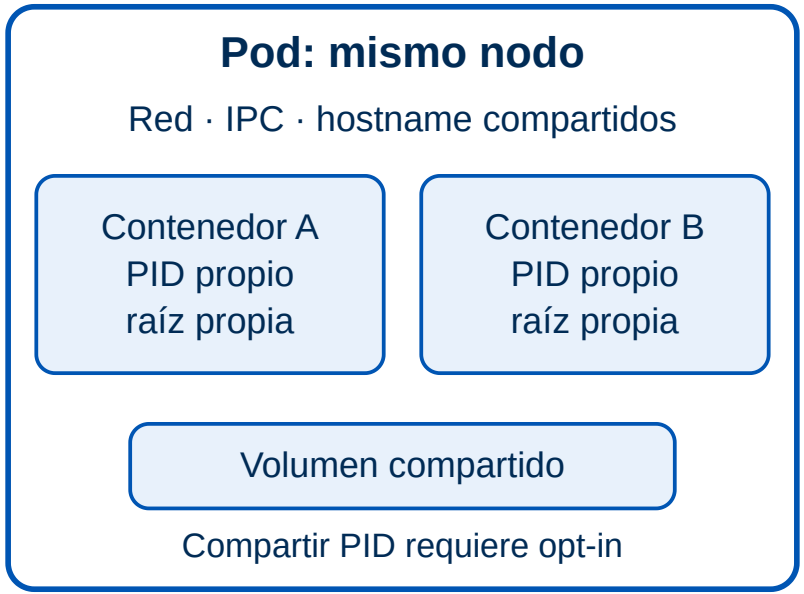


Fuente: Kubernetes · contexto del Pod

¿Qué comparte un Pod?

- **Red:** IP y puertos; comunicación por `localhost`.
- **IPC y hostname (UTS):** normalmente compartidos.
- **Procesos (PID):** separados por defecto; compartir requiere `shareProcessNamespace: true`.
- **Filesystem:** raíz propia; volúmenes compartibles.

Los UID/GID de los procesos se configuran con `securityContext`.



Fuente: Kubernetes · compartir procesos

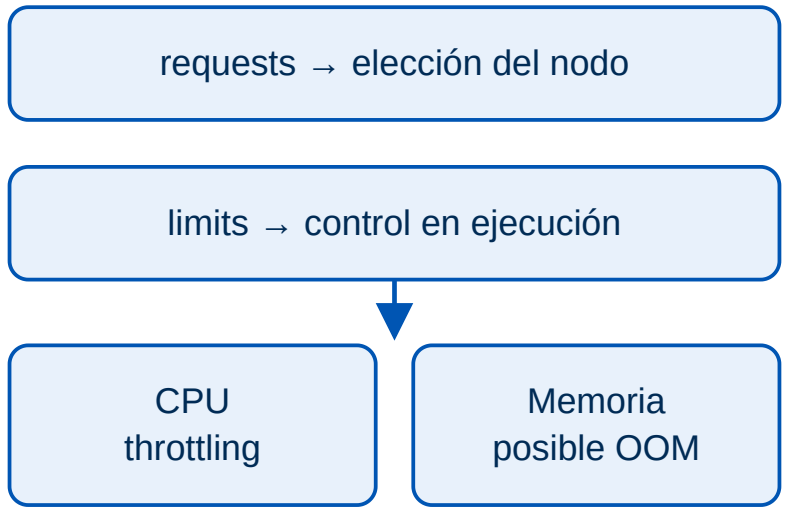
Recursos del Pod

Por contenedor, se declaran CPU y memoria:

- **requests:** demanda que el scheduler considera al elegir un nodo.
- **limits:** restricciones de ejecución; CPU puede sufrir throttling y memoria, OOM.

Linux aplica el control mediante **cgroups**; cgroup v2 es la generación actual.

El almacenamiento efímero se mide en bytes, no en throughput. Disco y red no tienen campos estándar de requests/limits para su velocidad.



Linux: cgroups (v2)

Fuente: Kubernetes · requests y limits

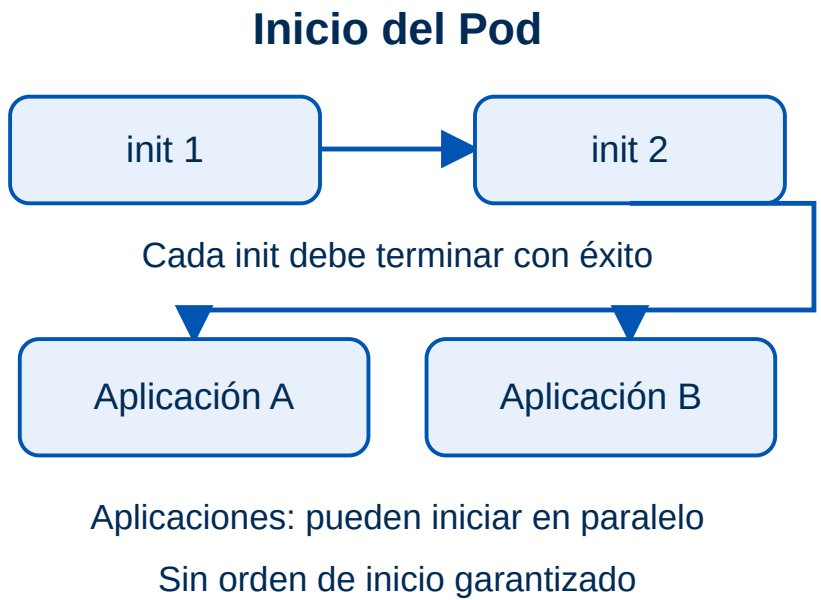
Contenedores de un Pod

Aplicación: ejecutan la lógica principal; pueden iniciar en paralelo.

Init: preparan el entorno. Los init comunes se ejecutan en orden y deben terminar con éxito.

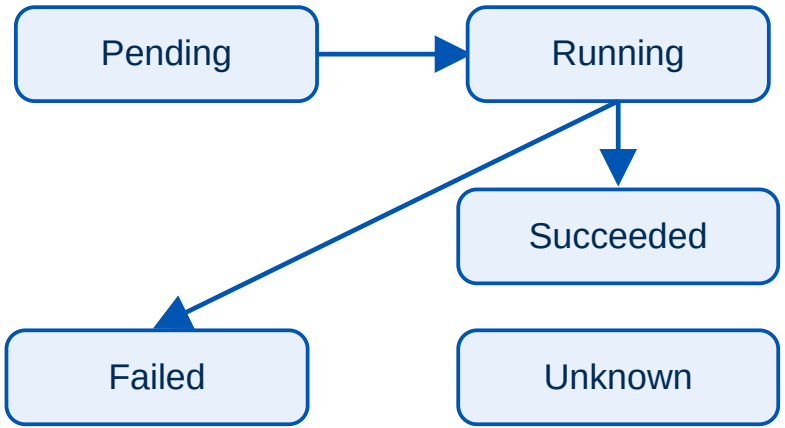
Sidecar nativo: auxiliar que permanece activo; se declara en `initContainers` con `restartPolicy: Always`.

Efímero: se agrega a un Pod existente para depurar; no se reinicia automáticamente.



Fuente: Kubernetes · init containers

Ciclo de vida del Pod



Unknown: estado no disponible

Running ≠ Ready

Running no implica Ready: la aplicación puede no estar preparada para recibir tráfico.

Fuente: Kubernetes · ciclo de vida

Fases del Pod

- **Pending:** aceptado; espera asignación o preparación de contenedores, incluida la descarga de imágenes.
- **Running:** asignado a un nodo; contenedores creados y al menos uno ejecutándose, iniciándose o reiniciándose.
- **Succeeded:** todos finalizaron con éxito y no se reiniciarán.
- **Failed:** todos terminaron; alguno falló y no se reiniciará.
- **Unknown:** no se pudo obtener el estado, normalmente por un problema de comunicación con el nodo.

`kubectl get pods` muestra una columna STATUS: `CrashLoopBackOff` y `Terminating` no son fases del Pod.

Fuente: Kubernetes · fases y estados

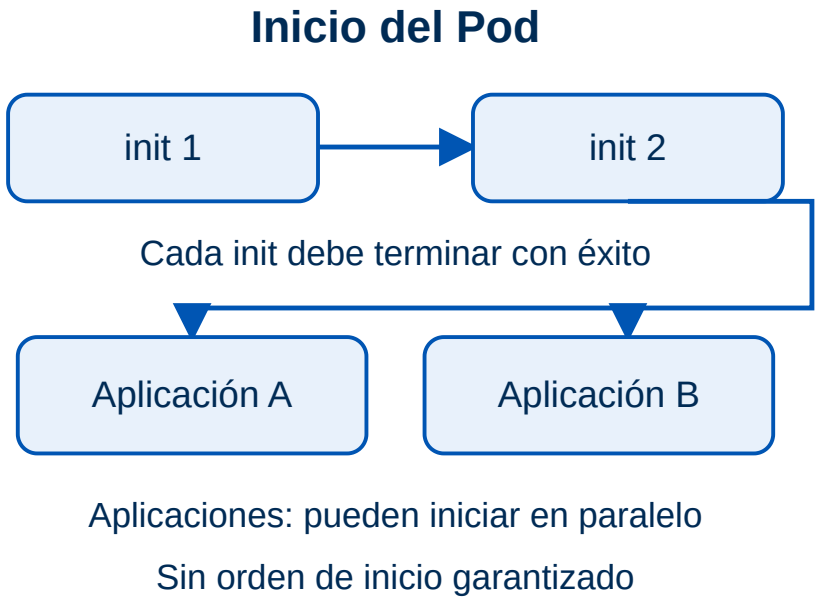
Ciclo de vida de los contenedores

Los estados de un contenedor son **Waiting**, **Running** y **Terminated**, distintos de la fase del Pod.

Las probes comprueban su salud:

- **startup:** comprueba el arranque; demora las otras probes hasta que tenga éxito.
- **readiness:** determina si está listo para recibir tráfico.
- **liveness:** detecta cuándo reiniciarlo.

Los init comunes completan la preparación antes de iniciar las aplicaciones.



Fuente: Kubernetes · probes

Lifecycle handlers

`postStart` puede ejecutarse junto con el ENTRYPOINT. El período de gracia comienza **antes** de `preStop` e incluye el hook y el apagado del proceso.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: lifecycle-demo
spec:
  terminationGracePeriodSeconds: 30
  containers:
  - name: web
    image: nginx:1.30.5
    lifecycle:
      postStart:
        exec:
          command: ["/bin/sh", "-c", "echo iniciado > /tmp/post-start"]
      preStop:
        exec:
          command: ["/bin/sh", "-c", "sleep 5"]
```

Fuente: Kubernetes · hooks

Init Container

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: myapp-container
    image: busybox:1.37.0
    command: ["sh", "-c", "echo 'La app está en ejecución'; sleep 3600"]
  initContainers:
  - name: init-myservice
    image: busybox:1.37.0
    command:
    - sh
    - -c
    - >-
      until nslookup myservice;
      do echo 'Esperando myservice'; sleep 2; done
```

El init espera que DNS resuelva `myservice` en el mismo namespace.

Fuente: Kubernetes · init containers

Init Container · continuación

```
# Continúa la lista spec.initContainers del Pod anterior:
  - name: init-mydb
    image: busybox:1.37.0
    command:
    - sh
    - -c
    - >-
      until nslookup mydb;
      do echo 'Esperando mydb'; sleep 2; done
```

`init-mydb` comienza cuando `init-myservice` termina con éxito. La aplicación comienza después de ambos.

Resolver DNS demuestra que el nombre existe; **no garantiza que el servicio o la base de datos estén listos.**

Fuente: Kubernetes · orden de inicialización

Pod Managers

En general, no creamos Pods aislados: declaramos recursos cuyos controladores gestionan su creación y reemplazo.

- **Deployment / ReplicaSet:** mantienen una cantidad de réplicas.
- **DaemonSet:** ejecuta un Pod por nodo elegible.
- **StatefulSet:** mantiene una identidad estable por Pod.

Fuente: Kubernetes · controladores de workloads

Pod Managers

Deployment

Un Deployment proporciona actualizaciones declarativas para Pods y ReplicaSets. Su controlador acerca el estado observado al deseado.

Permite reemplazar progresivamente una versión de la aplicación por otra.

Un **rollout** se dispara cuando cambia `.spec.template`, por ejemplo, la imagen del contenedor. Cambiar sólo `.spec.replicas` escala la aplicación sin iniciar un rollout.

Fuente: Kubernetes · Deployment y rollout

Deployment · manifiesto

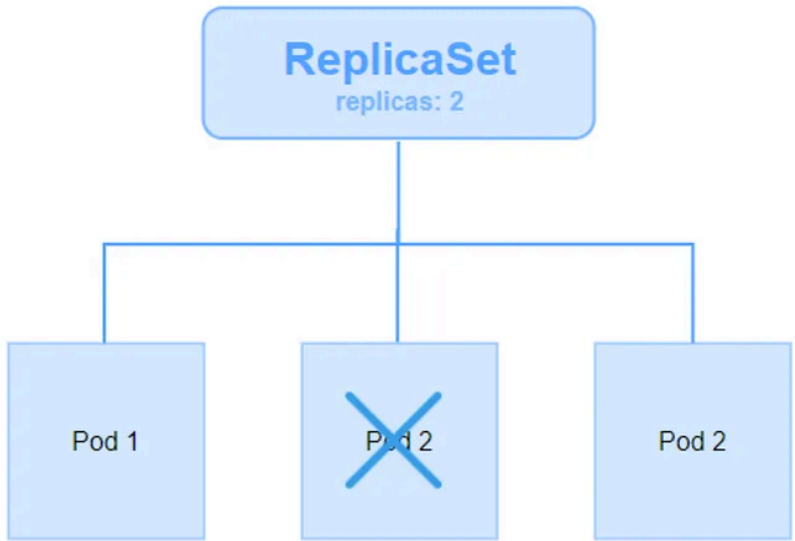
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.30.5
        ports:
        - containerPort: 80
```

ReplicaSet

Mantiene la cantidad deseada de Pods: crea los que faltan y elimina los que sobran.

Identifica sus Pods mediante un selector de etiquetas y las referencias de ownership.

Puede adoptar un Pod ajeno al template si coincide con el selector y **no tiene otro controlador propietario**.



Fuente: Kubernetes · ReplicaSet

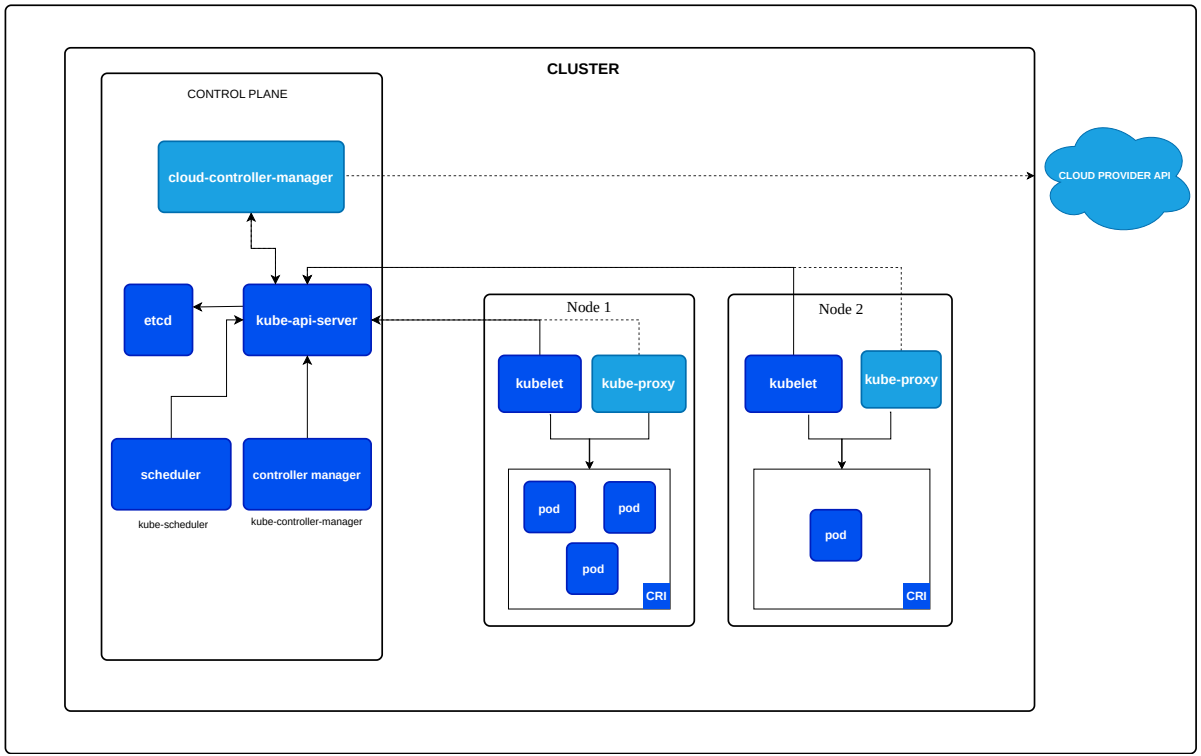
Deployment vs ReplicaSet

Deployment	ReplicaSet
Gestiona los ReplicaSets de una aplicación.	Mantiene una cantidad de Pods.
Cambiar el template inicia un rollout.	El template se usa al crear nuevos Pods.
Coordina la sustitución entre versiones.	No actualiza los Pods existentes al cambiar su template.
Permite volver a una revisión anterior.	Crea o elimina Pods para alcanzar `.spec.replicas`.

Para desplegar una aplicación, normalmente usamos un **Deployment**.

Fuente: Kubernetes · cuándo usar ReplicaSet

Plano de control y kubelet



Los objetos se gestionan mediante la API; el kubelet ejecuta sus contenedores, no los controladores.

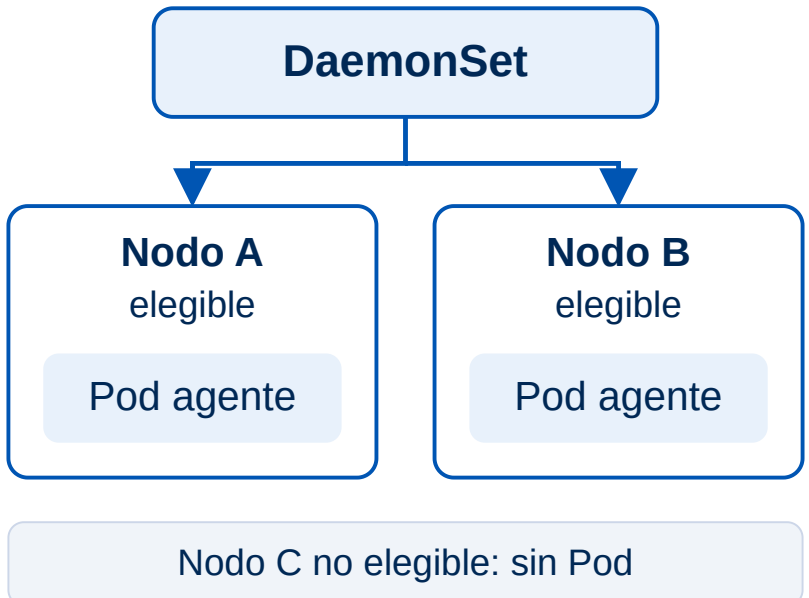
Fuente: Kubernetes · arquitectura del clúster

DaemonSet

Mantiene un Pod por **nodo elegible** para prestar un servicio local al nodo.

Al incorporarse un nodo elegible, crea un Pod; si uno se elimina, lo repone.

La elegibilidad depende del selector, la afinidad y las tolerations. Puede incluir nodos de control plane si la configuración lo permite.



Fuente: Kubernetes · DaemonSet

DaemonSet

Ejemplos de servicios que necesitamos en cada nodo elegible:

- Recolección de logs.
- Monitoreo del nodo.
- Agentes de red o seguridad.
- Servicios de almacenamiento local.

Su cantidad de Pods depende de los nodos elegibles, no de un campo `replicas`.

Fuente: Kubernetes · usos de DaemonSet

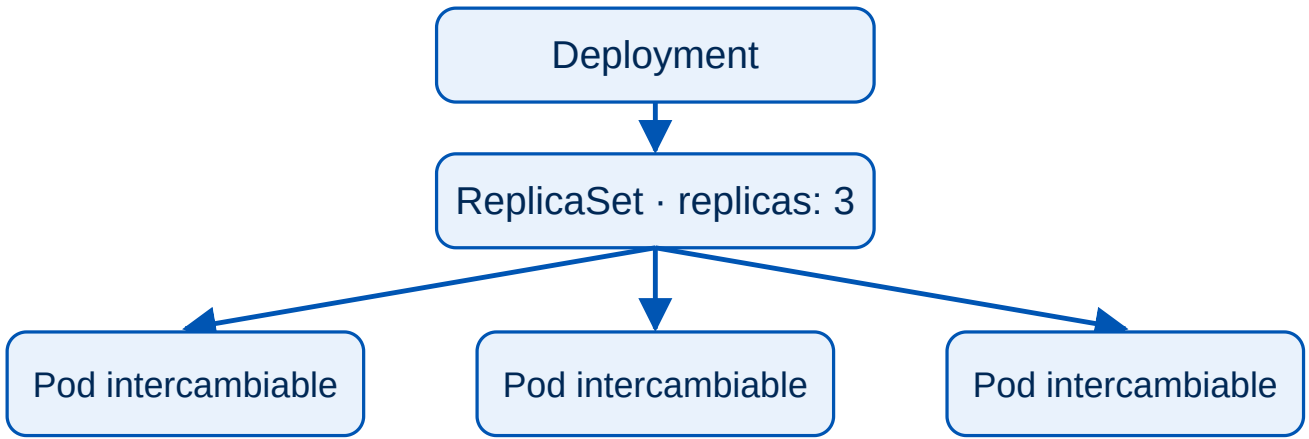
StatefulSet

- Mantiene una **identidad estable por Pod**, incluso al reemplazarlo.
- Cada Pod tiene un ordinal y un nombre: `web-0`, `web-1`, `web-2`.
- Un Service **headless** proporciona la identidad de red estable.
- Si configuramos `volumeClaimTemplates`, cada Pod obtiene sus propios PVCs para conservar su almacenamiento.
- La identidad pertenece al Pod; puede ejecutarse en otro nodo.

Fuente: Kubernetes · identidad en StatefulSet

Deployment vs StatefulSet vs DaemonSet

Deployment: cantidad de réplicas

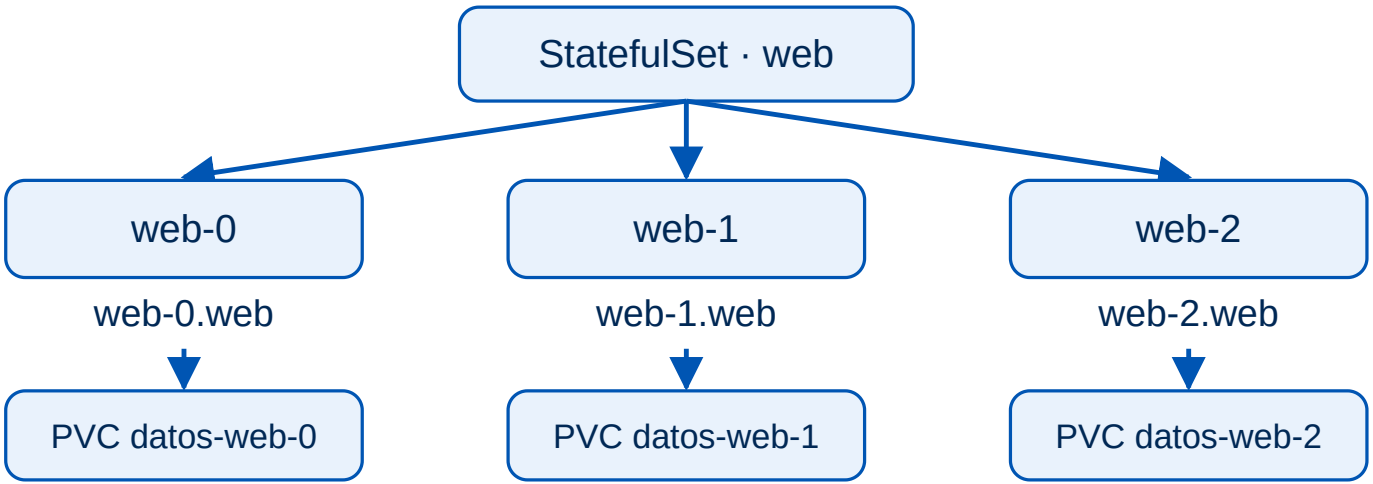


El scheduler elige los nodos; no hay una identidad ordinal estable.

Fuente: Kubernetes · Deployment

Deployment vs StatefulSet vs DaemonSet

StatefulSet: identidad estable por Pod

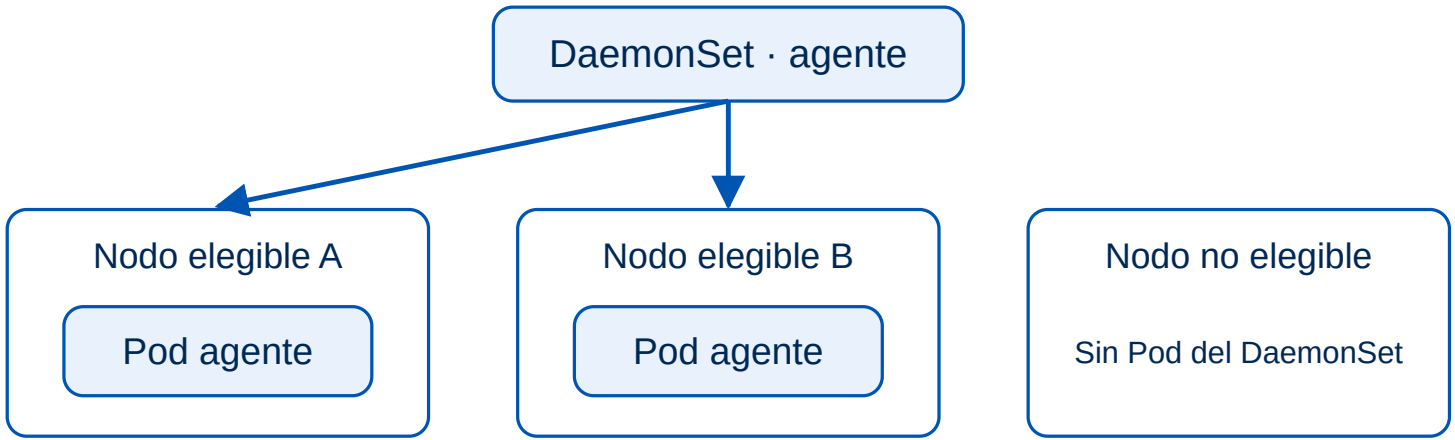


Almacenamiento ilustrado con volumeClaimTemplates (opcional).

Fuente: Kubernetes · StatefulSet

Deployment vs StatefulSet vs DaemonSet

DaemonSet: un Pod por nodo elegible



Elegibilidad: selector / afinidad y tolerations de los Pods.

Fuente: Kubernetes · DaemonSet

Job

Un Job ejecuta una tarea hasta alcanzar la cantidad prevista de finalizaciones exitosas.

- `.spec.completions`: cantidad de Pods que deben terminar con éxito.
- `.spec.parallelism`: cantidad de Pods que se solicita ejecutar en paralelo.

Por ejemplo: `completions: 6` y `parallelism: 2` solicitan seis finalizaciones, normalmente con hasta dos Pods activos a la vez.

Los reintentos están limitados por `.spec.backoffLimit`; `.spec.activeDeadlineSeconds` permite limitar el tiempo total.

Fuente: Kubernetes · Jobs

CronJob

Un CronJob crea Jobs según una programación con sintaxis cron.

- `.spec.schedule`: cuándo ejecutar; `.spec.timeZone`: zona horaria, por ejemplo `Etc/UTC`.
- `.spec.concurrencyPolicy`: permite ejecuciones simultáneas (`Allow`), evita solaparlas (`Forbid`) o reemplaza la anterior (`Replace`) para ese CronJob.

La programación es **aproximada**: puede haber ejecuciones duplicadas u omitidas. La tarea debe ser **idempotente**: repetirla no debe duplicar sus efectos.

Fuente: Kubernetes · CronJobs

Reconciliation

Reconciliación

Los controladores comparan el estado observado con el deseado y actúan para acercarlos.

La convergencia necesita tiempo y puede verse impedida por falta de recursos o errores.

Tener o no campos `spec` y `status` **no determina** si un recurso puede ser observado o gestionado por controladores.

El formato no determina si hay reconciliación

Deployment

spec: estado deseado
status: estado observado

ConfigMap / Secret

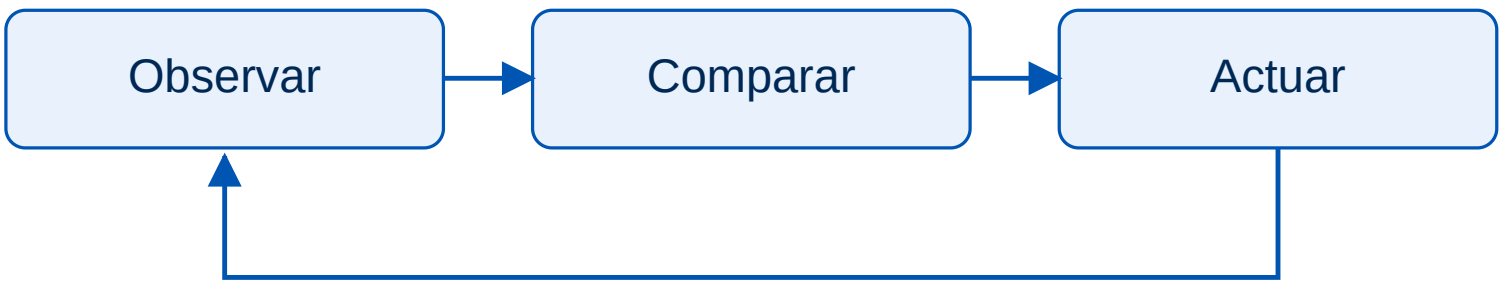
data: configuración
Pueden ser observados o gestionados por controladores

Los controladores actúan sobre objetos.

Fuente: Kubernetes · controladores

Reconciliación

Reconciliación: acercar lo observado a lo deseado



Deseado: 3 Pods · Observado: 2 → crear un reemplazo

El ciclo continúa; la convergencia depende de que la acción sea posible.

Fuente: Kubernetes · bucle de control

Proceso de reconciliación

1. El controlador observa recursos a través de la API.
2. Compara lo observado con el estado deseado.
3. Solicita acciones mediante la API: crear, actualizar o eliminar recursos.
4. Otros componentes realizan su parte y reportan lo observado.
5. El ciclo se repite; los resultados no son instantáneos.

Cambiar un ConfigMap no inicia por sí solo un rollout del Deployment: para ello debe cambiar el template del Pod.

Fuente: Kubernetes · actualización de ConfigMaps

Proceso de reconciliación

Queremos tres Pods de nginx. Declaramos un **Deployment** con `replicas: 3` y el template del Pod.

El Deployment controller gestiona un **ReplicaSet**, cuyo controlador crea los Pods necesarios. No tenemos que crear el ReplicaSet manualmente.

Fuente: Kubernetes · Deployment

Proceso de reconciliación · manifiesto

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.30.5
        ports:
        - containerPort: 80
```

Proceso de reconciliación

Aplicamos el manifiesto y esperamos el despliegue:

```
kubectl apply -f nginx-deployment.yaml
kubectl rollout status deployment/nginx-deployment
kubectl get pods -l app=nginx
```

El primer comando guarda la intención. Los controladores crean los recursos y el kubelet pone en marcha sus contenedores.

Fuente: Kubernetes · desplegar una aplicación

Proceso de reconciliación · Pod → ReplicaSet

Seleccionamos un Pod del ejemplo y consultamos su controlador:

```
pod=$(kubectl get pods -l app=nginx \
  -o jsonpath='{.items[0].metadata.name}')
kubectl get pod "$pod" -o json | \
  jq '.metadata.ownerReferences[] | select(.controller == true)'
```

Extracto ilustrativo; los nombres y UID dependen del clúster:

```
{
  "kind": "ReplicaSet",
  "name": "nginx-deployment-<hash>",
  "uid": "<uid-del-replicaset>",
  "controller": true
}
```

Fuente: Kubernetes · owners y dependents

Proceso de reconciliación · ReplicaSet → Deployment

Seguimos la referencia al ReplicaSet real:

```
rs=$(kubectl get pod "$pod" -o json | \
  jq -r '.metadata.ownerReferences[] | select(.controller == true) | .name')
kubectl get rs "$rs" -o json | \
  jq '.metadata.ownerReferences[] | select(.controller == true)'
```

Extracto ilustrativo:

```
{
  "kind": "Deployment",
  "name": "nginx-deployment",
  "uid": "<uid-del-deployment>",
  "controller": true
}
```

Fuente: Kubernetes · cadena de ownership

Consistencia eventual

Al crear un Namespace, el plano de control también asegura que exista un ServiceAccount llamado `default`.

Estas acciones no forman una única operación instantánea: el ServiceAccount puede aparecer después de que la creación del Namespace haya terminado.

```
kubectl create namespace fede-ns
kubectl get serviceaccounts -n fede-ns --watch
# Luego aparece el ServiceAccount default.
# Ctrl+C para dejar de observar.
```

Fuente: Kubernetes · ServiceAccount controller

Consistencia eventual

1. `kubectl` solicita crear el Namespace al **API server**, que lo persiste en etcd.
2. El **serviceaccount-controller** observa el Namespace activo.
3. Si falta `default`, solicita crear ese ServiceAccount mediante la API.
4. La API lo persiste y las consultas posteriores pueden observarlo.

El estado del clúster converge mediante acciones asíncronas. Este intervalo no significa que etcd sea una base de datos inconsistente.

Fuente: Kubernetes · creación de default

Consistencia eventual

Consultamos los dos objetos después de la convergencia:

```
kubectl get namespace fede-ns -o json | \
  jq '{kind, name: .metadata.name, phase: .status.phase}'
kubectl get serviceaccount default -n fede-ns -o json | \
  jq '{kind, name: .metadata.name, namespace: .metadata.namespace}'
```

Salida ilustrativa:

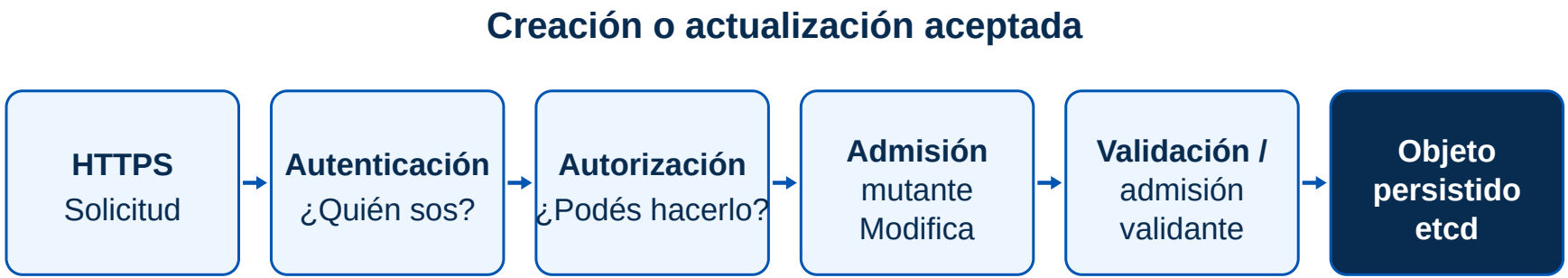
```
{"kind":"Namespace","name":"fede-ns","phase":"Active"}
{"kind":"ServiceAccount","name":"default","namespace":"fede-ns"}
```

El ServiceAccount no necesita campos `spec` y `status` para ser gestionado por un controlador.

Fuente: Kubernetes · objetos gestionados por controladores

Acceso a la API

De la API hasta el etcd



Vista simplificada: se omiten conversión y valores por defecto (defaulting).
get / list / watch: sin admisión y sin persistencia de objetos.

Fuente: Kubernetes · acceso y admisión

Service Accounts

- Un `ServiceAccount` identifica a una aplicación para autenticarse ante la API; los permisos se autorizan por separado.
- Con RBAC, `Role` y `RoleBinding` conceden los permisos mínimos necesarios.
- Un Pod sin cuenta explícita usa `default` de su namespace, creada por un controlador.
- Por defecto, los Pods reciben tokens de corta duración, proyectados en un volumen y renovados automáticamente.

```
kubectl get serviceaccounts -o wide
# NAME      SECRETS   AGE
# default   0         27h
```

SECRETS `0` es normal: desde 1.24 no se crean automáticamente Secrets con tokens permanentes.

Fuente: Kubernetes · ServiceAccounts y tokens

Admission Controllers

Después de autenticar y autorizar una solicitud, la admisión puede modificarla o rechazarla antes de persistir el objeto.

- **Mutante:** modifica el objeto; por ejemplo, un webhook puede inyectar un sidecar.
- **Validante:** comprueba reglas y puede rechazar el objeto, sin modificarlo.
- Hay controladores integrados en el API server y extensiones mediante webhooks; también se pueden definir políticas declarativas.

Se aplican a creación, modificación, eliminación y ciertas conexiones; `get`, `list` y `watch` no pasan por admisión.

Fuente: Kubernetes · control de admisión

Manifiestos

Un manifiesto YAML o JSON describe un recurso. `apiVersion` indica la versión de su API y `kind`, el tipo de objeto; **no son la versión del clúster**.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.30.5
    ports:
    - containerPort: 80
```

`v1` para Pods y `apps/v1` para Deployments siguen vigentes.

Fuente: Kubernetes · objetos y manifiestos

Namespaces

Un namespace organiza recursos y define un ámbito para sus nombres. **No impone aislamiento de seguridad ni de red por sí solo.**

- Pods, Services y PersistentVolumeClaims (PVC) pertenecen a un namespace.
- Nodes y PersistentVolumes (PV) tienen alcance de clúster.

```
kubectl get namespaces
# NAME              STATUS   AGE
# default           Active   30m
# kube-node-lease   Active   30m
# kube-public       Active   30m
# kube-system       Active   30m
```

Estos son los cuatro namespaces iniciales; otros dependen de las aplicaciones y componentes instalados.

Fuente: Kubernetes · Namespaces

Fin!!